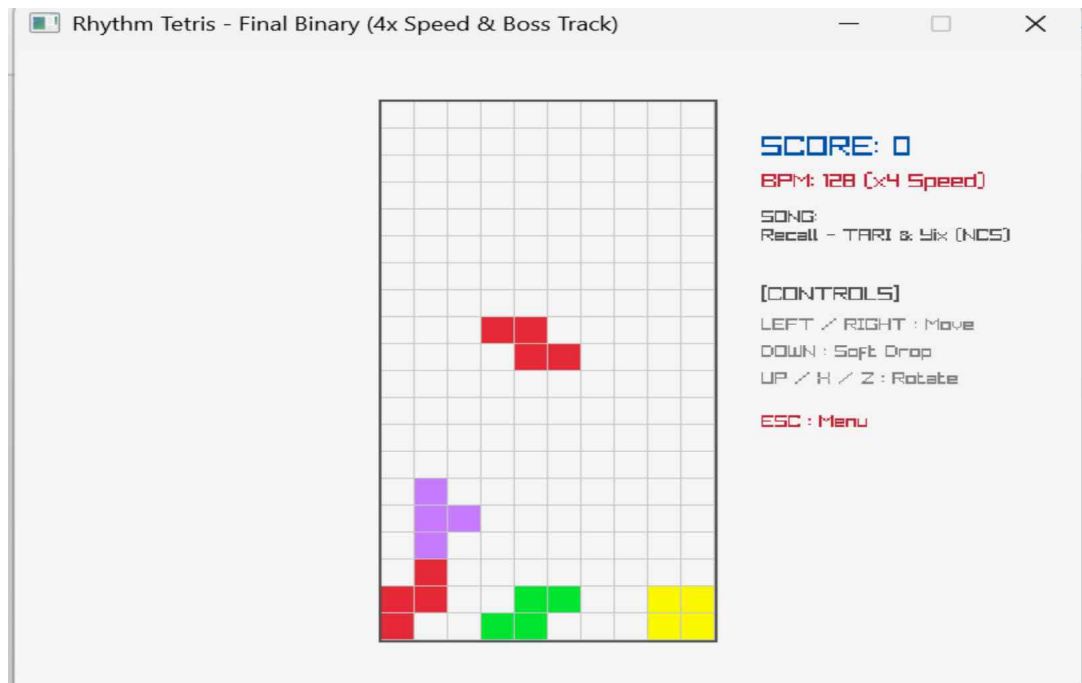


# 테트리스 게임 공격

정명훈

## 1.게임 실행



게임을 실행하니 3가지 단계가 있는 테트리스 게임이라는 것을 파악했고 bpm에 따라서 블록이 떨어지는 속도가 결정된다는 것을 파악했고 나의 목표를 점수 조작과 블록이 떨어지는 속도 조작 그리고 특정 블록만 떨어지게 하는 것으로 잡았다.

## 2.Ida를 이용한 기초적이 정적 분석

| Address          | Ordinal | Name             |
|------------------|---------|------------------|
| USER32           |         |                  |
| 000000014027F3A0 |         | IsIconic         |
| 000000014027F3A8 |         | BringWindowToTop |
| 000000014027F3B0 |         | IsZoomed         |
| 000000014027F3B8 |         | SetClipboardData |
| 000000014027F3C0 |         | EmptyClipboard   |
| 000000014027F3C8 |         | SetFocus         |
| 000000014027F3D0 |         | GetActiveWindow  |
| 000000014027F3D8 |         | GetClipboardData |

| Address            | Length   | Type | String                                   |
|--------------------|----------|------|------------------------------------------|
| .rdata:00000000... | 00000024 | C    | Target Binary for Reversing Project      |
| .rdata:00000000... | 00000015 | C    | > SELECT SONG & PLAY                     |
| .rdata:00000000... | 00000015 | C    | SELECT SONG & PLAY                       |
| .rdata:00000000... | 0000000A | C    | > CREDITS                                |
| .rdata:00000000... | 0000000A | C    | CREDITS                                  |
| .rdata:00000000... | 00000007 | C    | > EXIT                                   |
| .rdata:00000000... | 00000007 | C    | EXIT                                     |
| .rdata:00000000... | 00000029 | C    | Use UP/DOWN to navigate, ENTER to select |

우선 import와 string부분부터 확인을 하니 두 부분다 잘 보이는 것으로 보아 패킹과 난독화는 적용되지 않았다는 사실을 파악했다.

그 다음에는 function에 ctrl + f로 흑시모를 안티디버깅 함수가 있나 확인했고 불행히도 있었다 따라서 함수를 호출하는 곳으로 가보니 다행이도 한 곳에서만 호출하고 있어서 xdbg64에서 어느 시점에서 호출하는지 확인하기 위해서 중단점을 걸고 실행을 해 봤더니 신기하게 정상실행 중에는 문제가 없어서 아마 오류와 예외처리 시에 디버거 감지와 같은 기능을 수행하는 것 같아서 일단 넘어갔다.

| CPU              | 로깅                 | 메모                                        | 중단점 | 메모리 맵 | 호출 스택 | SEH | 스크립트 |
|------------------|--------------------|-------------------------------------------|-----|-------|-------|-----|------|
| 00007FF64E624E6F | FF15 93A30800      | call qword ptr ds:[<&IsDebuggerPresent> ] |     |       |       |     | RAX  |
| 00007FF64E624E75 | 85C0               | test eax, eax                             |     |       |       |     | RBX  |
| 00007FF64E624E77 | 0F85 3D010000      | jne thym_tetris.7FF64E624F8A              |     |       |       |     | RCX  |
| 00007FF64E624E7D | 48:8D8424 60020000 | lea rax, qword ptr ss:[rsp+260]           |     |       |       |     | RDY  |
| 00007FF64E624E85 | C74424 28 04010000 | mov dword ptr ss:[rsp+28], 104            |     |       |       |     |      |

```

37 }
38 if ( !ErrorFunc && !ErrorFuncW || v12 && IsDebuggerPresent() )
39     goto LABEL_23;
40 _RTC_GetSrcLine((unsigned __int8 *)retaddr - 5, srcNameW, 0x104u, lineNum, moduleNameW, 0x104u);
41 if ( ErrorFuncW )
42 {
43     LODWORD(cchWideChar) = v4;
44     if ( ErrorFuncW(crttype, srcNameW, lineNum[0], moduleNameW, L"Run-Time Check Failure #%- %s", cchWideChar, v11) != 1 )
45         return;
46 LABEL_23:
47     debugbreak();

```

그 다음에는 string부분에 점수를 출력하는 부분으로 이동하니

```
515 v62 = -16713219;
516 v8 = j_TextFormat("FINAL SCORE: %d");
517 i DrawText(v8, 300, 280, 22, v62);
```

j\_TextFormat가 있는데 뭔지 몰라서 검색 후 알아봤더니 이는 TextFormat 함수를 호출하는 함수고 TextFormat함수는 일종의 raylib의 표준 함수라는 것을 구글링을 통해서 파악을 했다. 그 다음에는 내가 조작하고 싶은 값인 점수가 인자로 어떻게 전달되는지를 확인하기 위해서 어셈블리로 코드를 보니

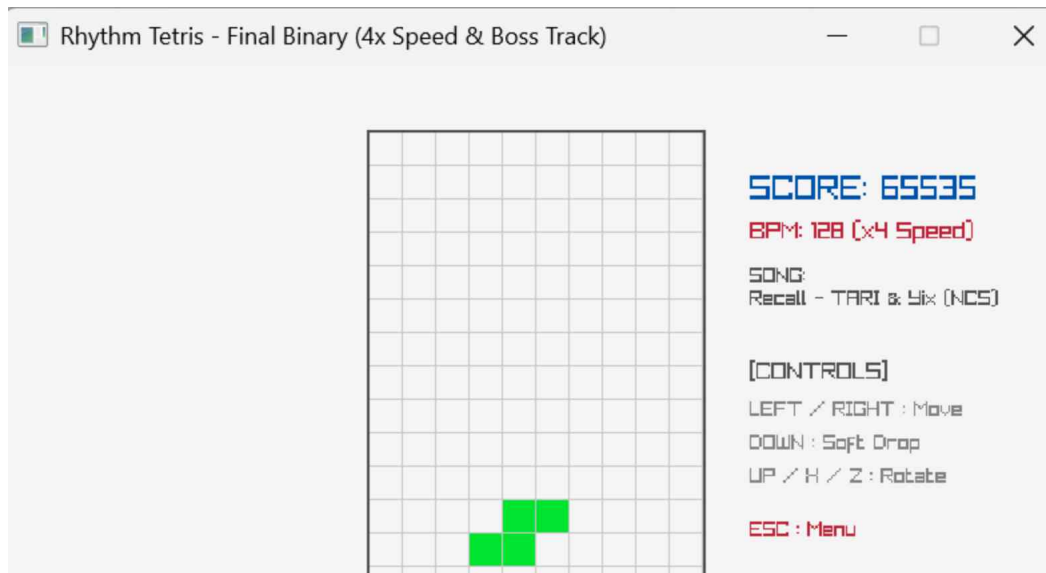
```
.text:00000001400AEED0 mov byte ptr [rbp+1550h+var_D8C+3], 0FFh
.text:00000001400AEED4 mov     edx, cs:score
.text:00000001400AEEDA lea     rcx, aFinalScoreD ; "FINAL SCORE: %d"
.text:00000001400AEED1 call    j_TextFormat
```

edx인자로 점수값을 받는 걸 확인했다. 여기서 score라는 값을 xref를 써서 추적해 보면

```
8
9 j__CheckForDebuggerJustMyCode(&_6A79DDDE_main_c);
10 for ( r = 0; r < 20; ++r )
11 {
12     for ( c = 0; c < 10; ++c )
13         board[r][c] = 0;
14 }
15 score = 0;
16 bpm = songList[selectedSongIndex].bpm;
17 if ( isMusicLoaded )
18 {
19     qmemcpy(v4, music, 0x38u);
```

이렇게 score를 초기화하는 데 심지어 bpm까지 조정하는 것을 찾아냈다  
이 지점에서 xdbg로 넘어가 패치를 하고 실행시

|                  |                      |                                   |
|------------------|----------------------|-----------------------------------|
| 00007FF64E4DD8D1 | EB B7                | jmp thychm_tetris.7FF64E4DD88A    |
| 00007FF64E4DD8D3 | C705 3FDE1A00 000000 | mov dword ptr ds:[<score>],0      |
| 00007FF64E4DD8DD | 48:6305 30DE1A00     | movsxd rax,dword ptr ds:[<selecte |
| 00007FF64E4DD8D1 | EB B7                | jmp thychm_tetris.7FF64E4DD88A    |
| 00007FF64E4DD8D3 | C705 3FDE1A00 FFFF00 | mov dword ptr ds:[<score>],FFFF   |
| 00007FF64E4DD8DD | 48:6305 30DE1A00     | movsxd rax,dword ptr ds:[<selecte |



점수가 시작하자마자 65535점이 나온다. 그 다음에 패치본rythem\_tetris\_v2로 저장했다.

그 다음에는 bpm을 조작하는 건데 나는 테트리스를 잘 못하기 때문에 속도를 아주 느리게 10정도로 할려고 우선 또 string에 가면 암호화가 하나도 안 되어 있어서 아주 쉽게

|                                                                                     |                   |          |   |                    |
|-------------------------------------------------------------------------------------|-------------------|----------|---|--------------------|
|  | .rdata:00007FF... | 0000000A | C | SCORE: %d          |
|  | .rdata:00007FF... | 00000013 | C | BPM: %d (x4 Speed) |
|  | .rdata:00007FF... | 00000000 | C | SONG: %s           |

이곳으로 가서 xref로 이동후

|                                                                                     |                        |     |                                          |
|-------------------------------------------------------------------------------------|------------------------|-----|------------------------------------------|
|  | .text:00007FF6DC78F3A3 | mov | byte ptr [rop+1550h+var_B1C+3], 0FFh     |
|  | .text:00007FF6DC78F3AA | mov | edx, cs:bpm                              |
|  | .text:00007FF6DC78F3B0 | lea | rcx, aBpmDX4Speed ; "BPM: %d (x4 Speed)" |
|  | .text:00007FF6DC78F3B7 | ... | ...                                      |

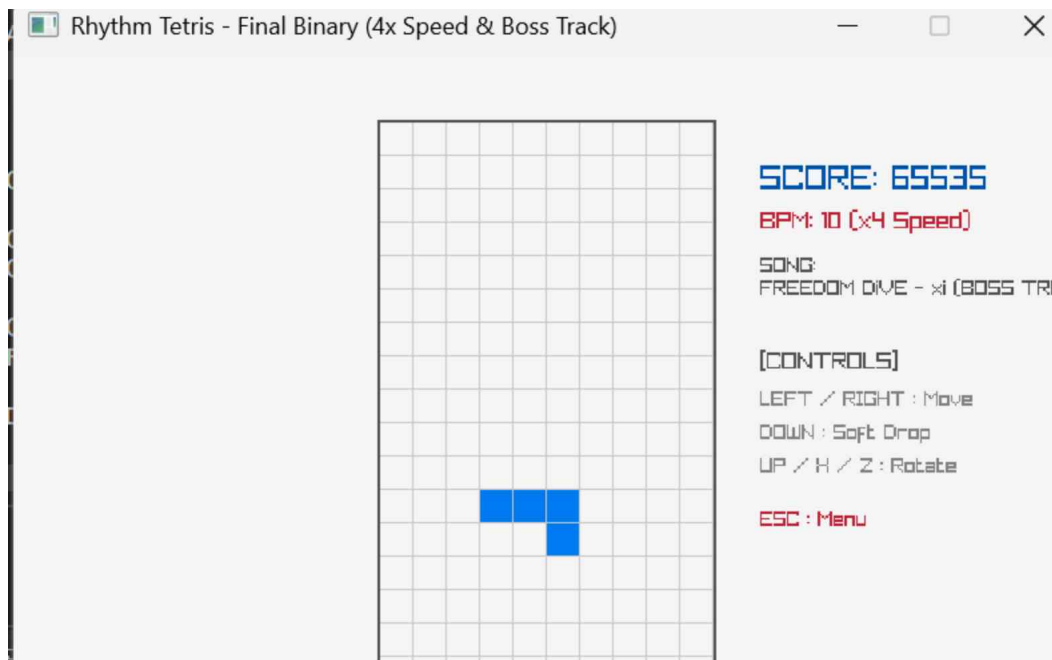
이 부분을 보면 또 너무 쉽게 bpm 전역변수로 선언하고 edx로 옮기고 있는 걸 발견할 수 있다. 그럼 또 bpm에 xref를 따라가면

|                                                                                     |                                        |
|-------------------------------------------------------------------------------------|----------------------------------------|
|  | score = 0;                             |
|  | bpm = songList[selectedSongIndex].bpm; |
|  | if ( isMusicLoaded )                   |

bpm조작하는 부분을 찾을 수 있는데 그냥 여기서 xdbg로 넘어가서 패치를 통해서 속도를 10으로 맞추고 실행을 하면

|                  |                  |                                   |
|------------------|------------------|-----------------------------------|
| 00007FF6DC78D8EF | 8B4401 10        | mov eax,dword ptr ds:[rcx+rax+10] |
| 00007FF6DC78D8F3 | 8905 57871A00    | mov dword ptr ds:[<bpm>],eax      |
| 00007FF6DC78D8F9 | 833D 4CE11A00 00 | cmp dword ptr ds:[<isMusicLoaded> |

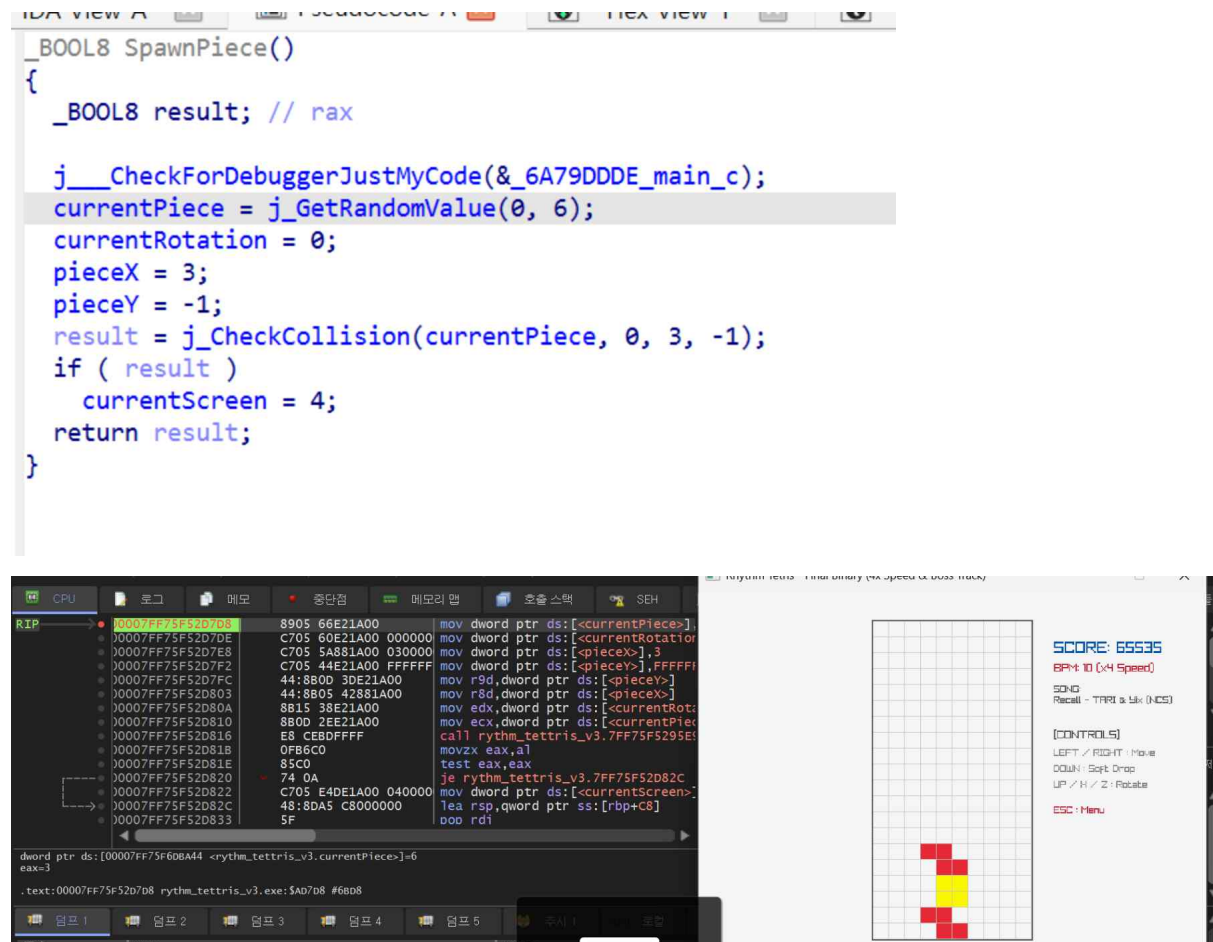
|                  |              |                             |
|------------------|--------------|-----------------------------|
| 00007FFC1F0E40FD | 837C24 50 0A | cmp dword ptr ss:[rsp+50],A |
| 00007FFC1F0E4102 | 90           | nop                         |
| 00007FFC1F0E4103 | 90           | nop                         |
| 00007FFC1F0E4104 | 90           | nop                         |
| 00007FFC1F0E4105 | 90           | nop                         |
| 00007FFC1F0E4106 | 90           | nop                         |
| 00007FFC1F0E4107 | 85C9         | test ecx,ecx                |



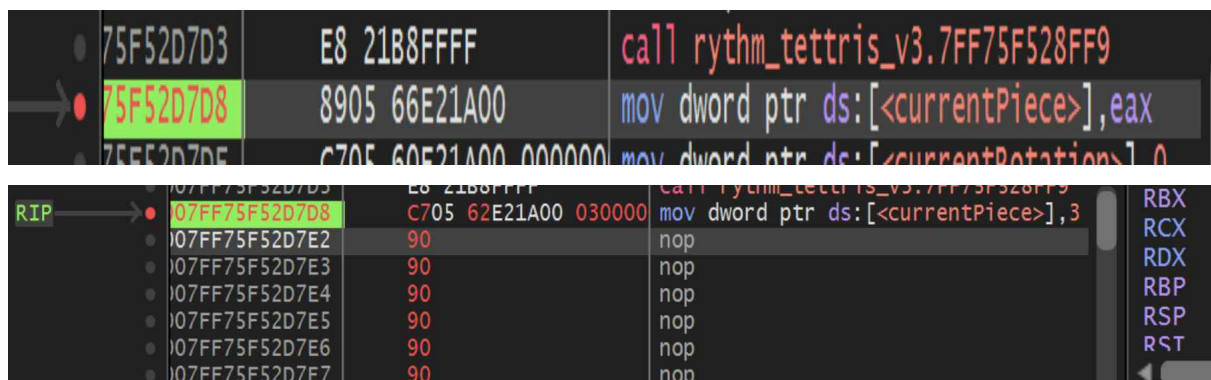
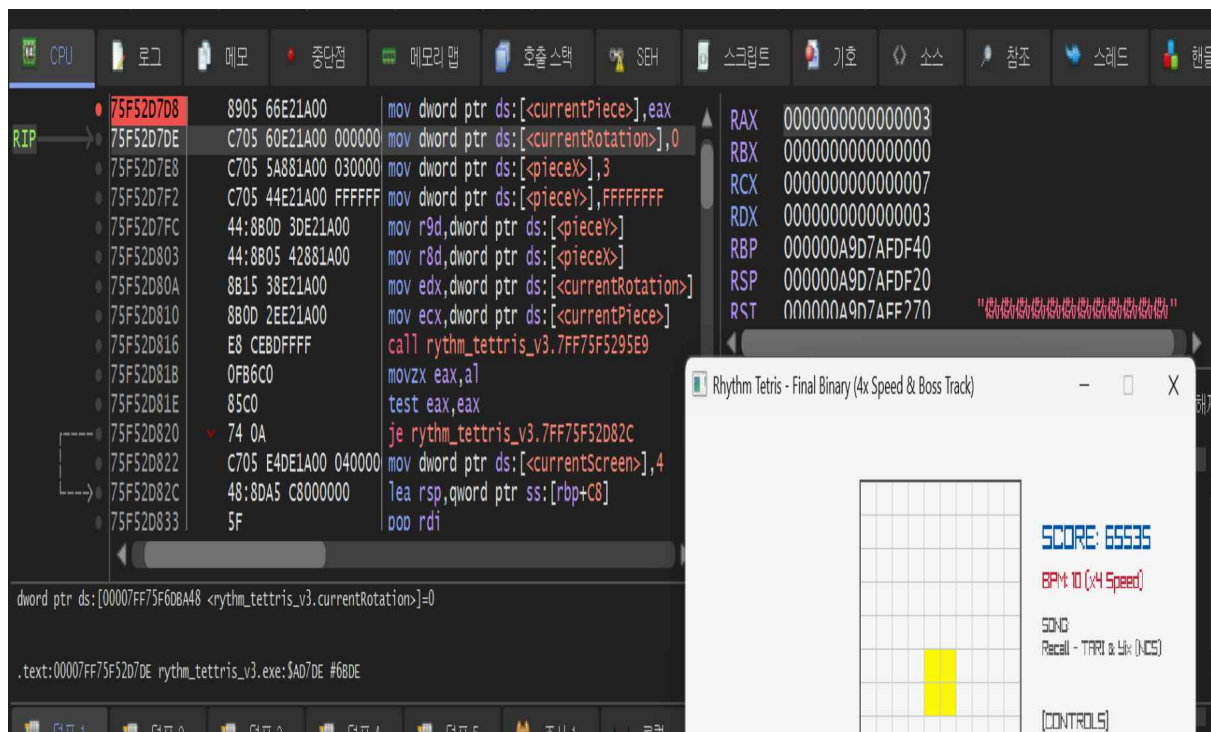
이렇게 bpm이 가장 빠른 단계를 선택했음에도 10이 출력되는 걸 확인할 수 있다. 이렇게 두 번째 목적까지 달성 완료하고 rythm\_tettris\_v3.exe에 패치를 반영해서 파일을 저장했다.

그 다음에는 나의 또다른 목표인 동일한 블록만 나오게 하는 것에서 나는 정사각형 2\*2크기의 블록만 나오게 하고 싶었다. 따라서 이전의 bpm과 점수를 초기화하는 코드가 보드를 초기화하는 코드니 거기 근처에서 한 번 블록을 생성하는 코드를 찾아 봤는데

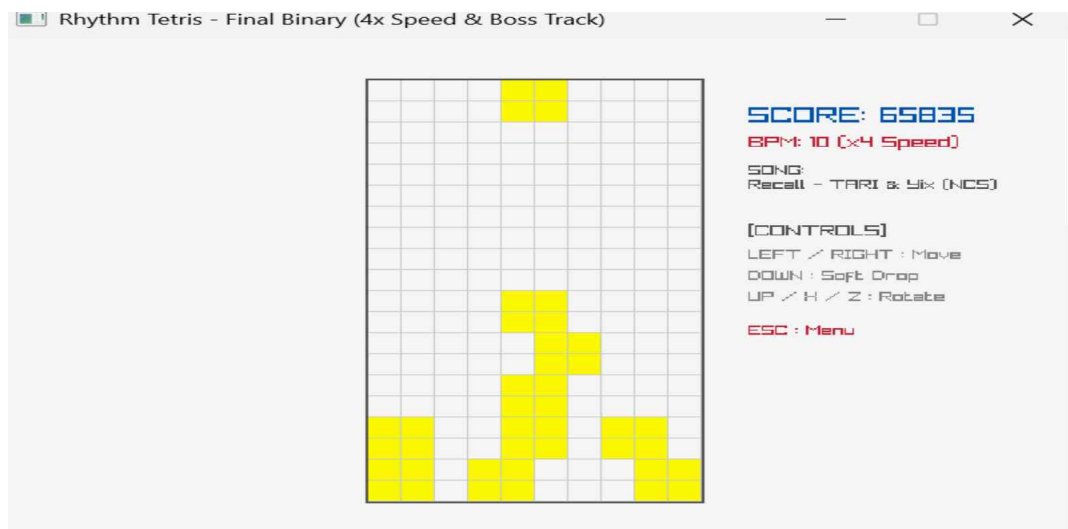
그렇게 찾던 중에 main을 xref로 하고 봤더니 GetBlockcell이라는 함수를 찾았다. 기대를 하고 같지만 그냥 회전에 따른 블록 위치를 변경하는 함수라는 걸 안 후에 다른 함수를 찾으러 다녔고 j\_SpawnPiece();를 발견하고 그 안에는 SpawnPiece함수가 존재했고 확인하니 블록을 생성하는 함수가 맞는 거 같았다. 그래도 혹시 모르니 xdbg64로 한 번 검증을 해보니



이런 식으로 currentPiece쪽에 매번 블록이 생성될 때마다 중단점이 걸리는 걸 보면서 확실히 이 함수 맞다라는 결론에 도달했고 그 다음에는 여기서 멈추는 게 아니라 그러면 내가 원하는 블록의 종류 정사각형일 때 currentPiece의 값이 뭔지를 알기 위해 블록을 정사각형일 때 레지스터 값 eax를 확인하니 정사각형일 때 eax는 3이라는 걸 알게 됐고 이를 xdbg64에서 패치를 하면



이렇게 되며 그 후 이걸 다시 v4로 저장하고 실행하면



이제는 오직 정사각형 블록만 나와 모든 단계를 매우 쉽게 클리어할 수 있게 되었다.

전반적으로 리버싱을 입문한 사람도 쉽게 접근할 수 있게 일부로 전역변수를 사용하고 함수 이름 암호화 역시 하나도 없었다는 점에서 기초 연습에 적합한 리버싱 게임이었다.

